Sentinelleai — Audit Report

Multisender.sol · 6dac8e9c-b4af-484d-97c2-3649384eed97 · 2026-05-22 19 h 53 min 38 s

CAUTION 72 / 100 5 consolidated findings

The Multisender contract is generally well-structured, using OpenZeppelin's SafeERC20, ReentrancyGuard, and Ownable2Step. The static-sentinel flagged a classic reentrancy pattern (REE-1/SC08) in withdrawFees(), but analysis by the historian agent and review of the code reveals that while the CEI pattern is technically violated (event emitted before call), the function is owner-only and the balance variable is a local snapshot — a reentrant call would find address(this).balance already drained by the in-flight transfer, making practical exploitation negligible. Nevertheless, the pattern warrants a LOW-MEDIUM advisory. The zero-address setter (ZAD-1/SC01) is intentional per developer documentation but represents a latent misconfiguration risk. Several formalist findings are false positives: Solidity 0.8.x provides built-in overflow protection (OVF-1/OVF-2 are invalid), withdrawToken has onlyOwner (ACL-1 is incorrect), and the duplicate zero-address medium finding collapses into ZAD-1. The token-address-is-a-contract validation concern is low-risk given SafeERC20 usage. Overall the contract triggers SC01 (Access Control / Input Validation) and SC08 (Reentrancy pattern) from OWASP Smart Contract Top 10 2026, but no finding rises to CRITICAL exploitability.

Severity breakdown

2 LOW 3 INFO

Static sentinel pre-scan

Static sentinels matched 2 pattern(s) — 1 HIGH, 1 LOW.

> **HIGH** `SC08 · REE-1 · line 141`
>
> **External call before state mutation (classic reentrancy)**
>
> Pattern call.value()() / .call{value:}() followed by state changes — checks-effects-interactions violated.
>
> ```
> (bool success, ) = to.call{value: balance}("");
> ```

> **LOW** `SC01 · ZAD-1 · line 40`
>
> **Setter accepts zero-address (no validation)**
>
> set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).
>
> ```
> function setFeeRecipient(address _feeRecipient) external onlyOwner { emit FeeRecipientUpdated(feeRecipient, _feeRecipient); feeRecipient = _feeRecipient; }
> ```

Consolidated findings

LOW CVSS 3.1 SC08: Reentrancy

REE-1: Checks-Effects-Interactions violation in withdrawFees()

The withdrawFees() function emits the FeesWithdrawn event and then makes an external ETH call without updating any state variable afterward. While the function is restricted to onlyOwner and the local `balance` snapshot means a reentrant call would find the contract balance already transferred (making practical exploitation extremely difficult), the pattern technically violates CEI and the static sentinel flagged it as HIGH confidence. The risk is further mitigated by the absence of any state mutation after the call. Nonetheless, best practice requires adherence to CEI.

```
withdrawFees(), line ~141: (bool success, ) = to.call{value: balance}("");
```

→ Emit the FeesWithdrawn event after the external call, or add the nonReentrant modifier to withdrawFees() as a defense-in-depth measure. Alternatively, restructure to set balance to zero in storage before the call (though no storage variable tracks this here).

Confirmed by: static-sentinel, developer, formalist

LOW CVSS 2.5 SC01: Access Control / Input Validation

ZAD-1: setFeeRecipient accepts zero address without explicit validation

The setFeeRecipient() function writes _feeRecipient directly to storage with no zero-address check. The developer intentionally allows address(0) as a signal to fall back to owner() in withdrawFees(), but this implicit behavior is undocumented in the function's NatSpec and could cause silent misconfiguration if called accidentally. The constructor also allows zero address for the same reason.

```
setFeeRecipient(), line ~40; constructor, line ~33
```

→ Either add require(_feeRecipient != address(0), 'Use owner() directly') and remove the fallback logic, or add explicit NatSpec documentation to setFeeRecipient() explaining that address(0) intentionally routes fees to owner(). The current comment is only on the state variable declaration.

Confirmed by: static-sentinel, developer, historian, formalist

INFO CVSS 0.0 SC10: Integer Overflow/Underflow

OVF-1/OVF-2: Integer overflow claims are false positives

The formalist agent flagged potential overflows in totalAmount and totalTokens accumulation loops. Solidity 0.8.x has built-in checked arithmetic that reverts on overflow by default. No SafeMath library is needed and no vulnerability exists.

```
multisendEther() accumulation loop; multisendToken() accumulation loop
```

→ No action required. Solidity 0.8.20 arithmetic is safe by default.

INFO CVSS 0.0 SC05: Access Control

ACL-1: withdrawToken missing access control — false positive

The formalist agent claimed withdrawToken has no access control. This is incorrect: the function is decorated with onlyOwner (line ~197 in context). No vulnerability exists.

```
withdrawToken(), onlyOwner modifier present
```

→ No action required.

INFO CVSS 0.0 SC04: Input Validation

VAL-1: Token address contract validation — negligible risk

The formalist agent flagged that multisendToken does not verify the token address is a deployed contract. SafeERC20's safeTransferFrom will revert on a non-contract address (call to EOA returns no data and the low-level call succeeds but safeTransferFrom checks return data). The require(token != address(0)) check is present. Risk is negligible in practice.

```
multisendToken(), line ~161
```

→ Optionally add an EXTCODESIZE check for defense-in-depth, but this is not a material vulnerability.

Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

CRITICAL `reentrancy` ethereum 2022-04-30 **$80 000 000**

Rari Capital/Fei Protocol — Flashloan Attack + Reentrancy (2022-04-30)

https://certik.medium.com/fei-protocol-incident-analysis-8527440696cc

CRITICAL `reentrancy` fantom 2021-12-18 **$30 000 000**

Grim Finance — Flashloan & Reentrancy (2021-12-18)

https://cointelegraph.com/news/defi-protocol-grim-finance-lost-30m-in-5x-reentrancy-hack

CRITICAL `reentrancy` ethereum 2020-04-19 **$25 000 000**

LendfMe — ERC777 Reentrancy (2020-04-19)

https://peckshield.medium.com/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dcc09

CRITICAL `access-control` ethereum 2025-05-28 **$12 000 000**

Corkprotocol — Access Control (2025-05-28)

https://x.com/SlowMist_Team/status/1928100756156194955

HIGH `access-control` bsc 2023-03-28 **$8 900 000**

SafeMoon Hack — Access Control (2023-03-28)

https://twitter.com/zokyo_io/status/1641014520041840640

HIGH `access-control` linea 2024-06-01 **$6 880 000**

VeloCore — lack-of-access-control (2024-06-01)

https://x.com/BeosinAlert/status/1797247874528645333

MEDIUM `bm25` 2022-11-01

stakehouse — Reentrancy - State access after external call

https://github.com/code-423n4/2022-11-stakehouse-findings/issues/50

HIGH `bm25` 2023-10-01

zksync — L1WethBridge.sol#L233 : possibility of reentrancy attack due to the state update after external call

https://github.com/code-423n4/2023-10-zksync-findings/issues/779

HIGH `bm25` 2023-09-01

ondo — Stealing extra mint fund by applying reentrancy attack on _execute with calling approve() again due to external call before crucial state update

https://github.com/code-423n4/2023-09-ondo-findings/issues/414

HIGH `bm25` 2023-03-01

asymmetry — An attacker can call back into the `withdraw()` function before it completes leading to reentrancy attacks.

https://github.com/code-423n4/2023-03-asymmetry-findings/issues/393

> **HIGH** `bm25` 2024-09-01
>
> kakarot — Unchecked External Call Success in execute_starknet_call and exec_precompile May Lead to Unexpected Behavior and Reentrancy Attacks
>
> https://github.com/code-423n4/2024-09-kakarot-findings/issues/115

> **MEDIUM** `bm25` 2023-09-01
>
> maia — invalid validation before call setLocalAddress function from CoreRootRouter.sol._setLocalToken
>
> https://github.com/code-423n4/2023-09-maia-findings/issues/269

Generated by Sentinelleai on 2026-05-22T23:58:00.369Z. Audit ID: `6dac8e9c-b4af-484d-97c2-3649384eed97`.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.